

Validation of ML Algorithms and Integration into Kubernetes Scheduler

Moving from Passive Observability to Active, Energy-Aware Orchestration



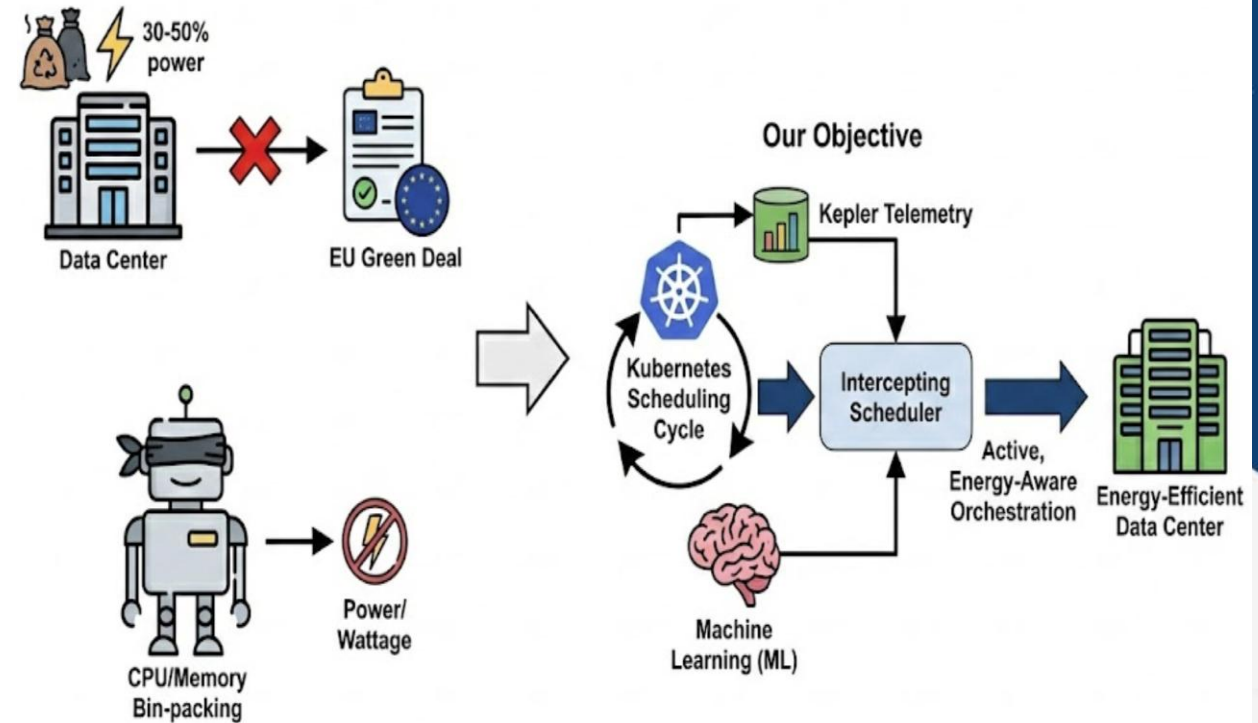
INSTITUT
POLYTECHNIQUE
DE PARIS

The Nokia logo, consisting of the word "NOKIA" in white capital letters on a blue-to-green gradient background.

NOKIA

The Problem & Our Objective

- The Macro Problem: Data centers waste 30-50% of their power due to inefficient workload placement, putting them at odds with strict regulations like the EU Green Deal.
- The Technical Flaw: Kubernetes orchestrates these workloads, but its default scheduler is completely blind to electrical power. It uses simple CPU/Memory bin-packing and ignores wattage.
- Our Objective: To intercept the Kubernetes scheduling cycle using Kepler telemetry and Machine Learning, moving from passive power monitoring to active, energy-aware orchestration.





Kubernetes: Quick Recap

From Hardware Boxes to Cloud Code

- Orchestrator: deploys and maintains containerized workloads (Pods) on Nodes
- Control plane: API server + etcd + controllers + scheduler
- Workers: kubelet + runtime; execute Pods and report the observed state
- Declarative: the desired state is expressed via the API; the system converges automatically

Simplified view

Control plane

scheduler

Worker node 1

kubelet & runtime
Pods

Worker node 2

kubelet & runtime
Pods



State-of-the-Art (Foundation & Observability)

Kepler Framework (Amaral et al., IEEE CLOUD 2023): * Leverages eBPF tracing and Intel RAPL sensors for real-time power estimation.

- Exports Prometheus-compatible metrics at node, pod, and container granularities.
- Achieves 95% accuracy with <1% overhead.
- Our Fit: Core scraping for our pipeline; enables baseline observability in deployed Grafana.

Container Energy Observability (Andrikopoulos, arXiv 2025): * Rigorous benchmarking of Kepler's attribution accuracy in multi-tenant setups.

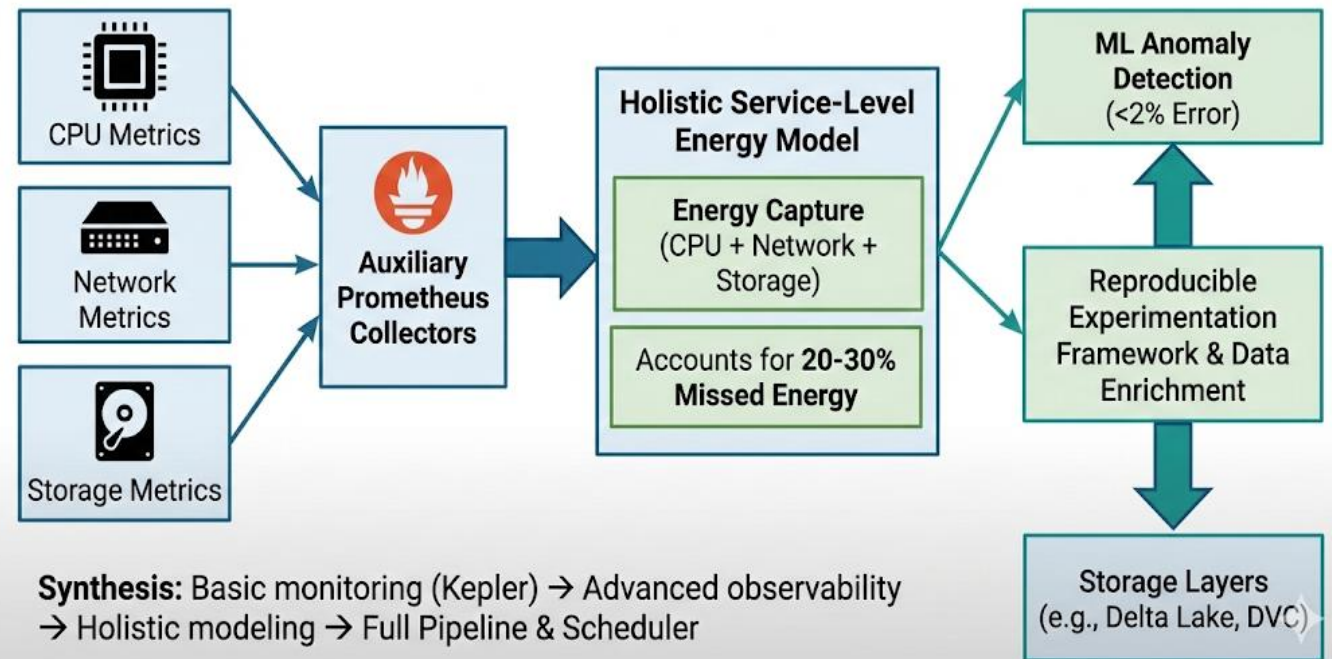
- Introduces KubeWatt: a hybrid ML model for dynamic/static power disaggregation using OpenTelemetry, yielding +15% precision.
- Our Fit: Metric validation and enrichment in our processing pipeline; provides reliable datasets for ML training on energy patterns.

State-of-the-Art (Holistic Modeling & Synthesis)

Service-Level Energy Modeling (Legler, arXiv 2025): * Holistic approach capturing CPU, network, and storage energy via auxiliary Prometheus collectors, accounting for 20-30% of total energy often missed.

- Provides a reproducible experimentation framework for microservice benchmarks, supporting ML anomaly detection with <2% error at scale.
- Our Fit: Informs data enrichment and storage layers (e.g., Delta Lake for ML-ready datasets); aligns with DVC versioning for research reproducibility.

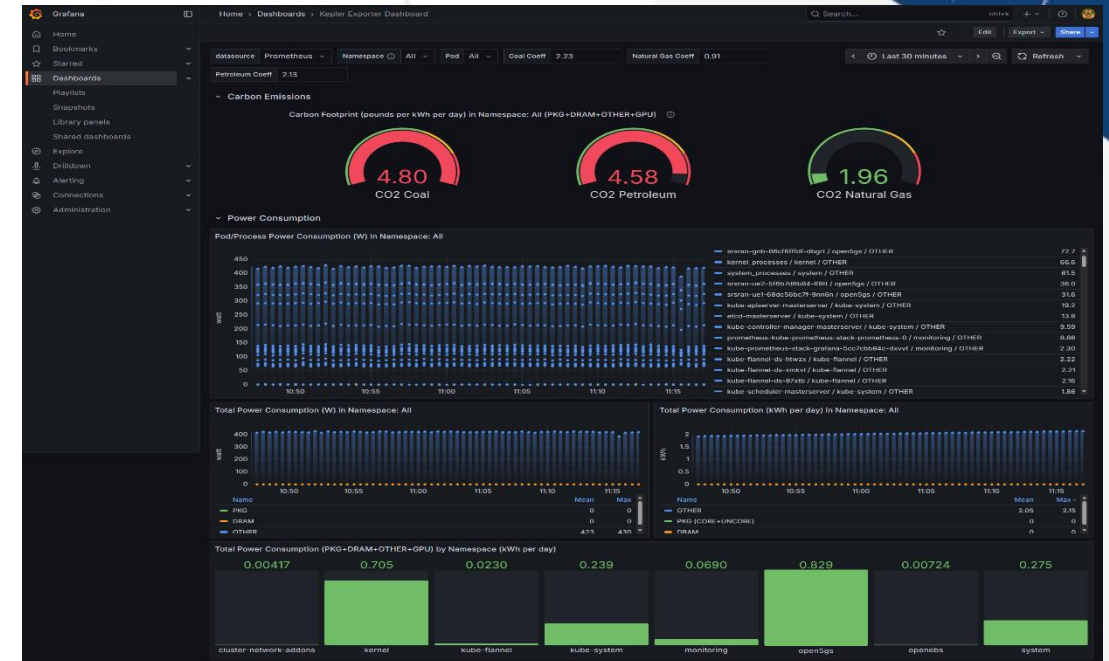
Holistic Service-Level Energy Modeling & Synthesis



Data Collection & Model Training

- Telemetry Stack: Kepler DaemonSet aggregated by kube-prometheus-stack.
- The Training Ground: Open5GS and srsRAN workloads used to generate highly variable, realistic network and compute stress.
- The Model: Tree-based ensemble trained on CPU, Memory, NetRX, NetTX, and Pod Type. Exported to ONNX.

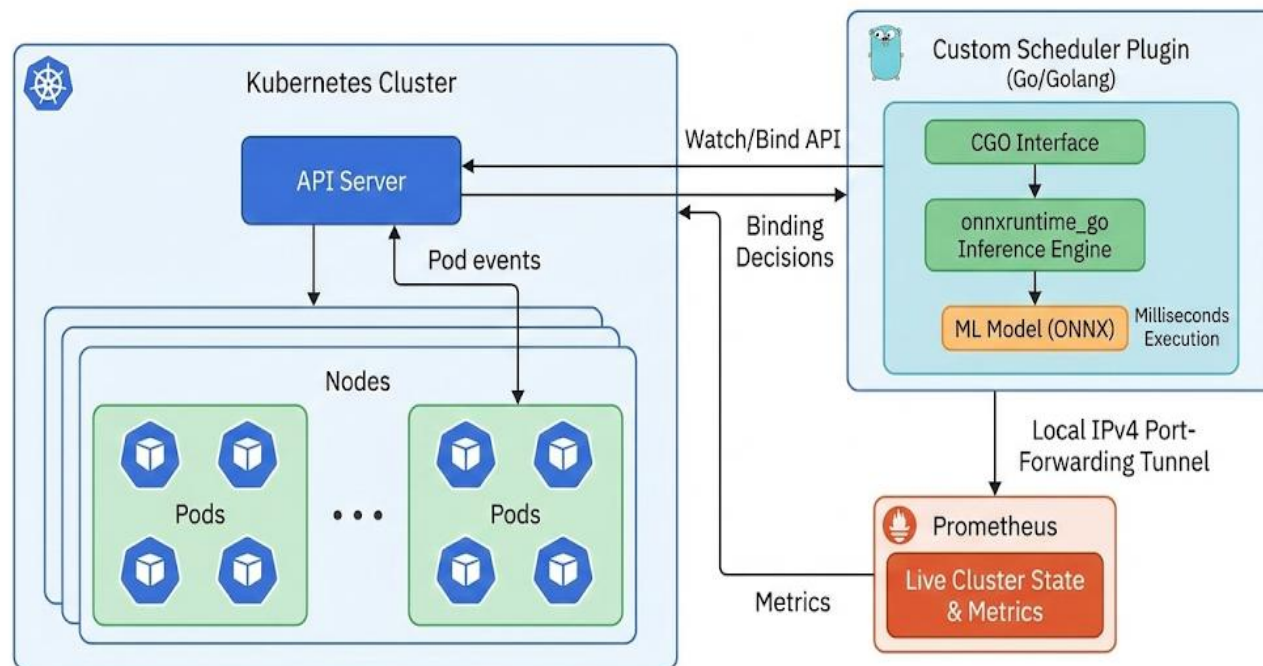
```
master@masterServer:~/ML_Integration_K8s_Scheduler$ cat open5gs-upf1-59d79bc876-75gmw_metrics.csv
timestamp,pod_type,cpu_cores,memory_bytes,net_rx_bytes_sec,net_tx_bytes_sec,energy_joules
2026-01-20T09:52:49,5G_CORE,0.0005880905511810868,21499904.0,10.118475824527698,10.118475824527698,0.0
2026-01-20T09:53:21,5G_CORE,8.514266972776644e-05,21508096.0,8.430232558139535,8.430232558139535,0.0
2026-01-20T09:53:37,5G_CORE,0.0,21508096.0,84.62819089900111,84.62819089900111,0.20700000000000074
2026-01-20T09:53:54,5G_CORE,0.0,21508096.0,126.55508815895462,126.55508815895462,0.04200000000000159
2026-01-20T09:54:10,5G_CORE,0.0,21508096.0,9.64336187546762,9.64336187546762,0.0
2026-01-20T09:54:42,5G_CORE,6.111243733587012e-05,21516288.0,0.0,0.0,0.0
2026-01-20T09:54:58,5G_CORE,0.0001754641427011356,21516288.0,8.067881485603005,8.067881485603005,0.0
2026-01-20T09:55:31,5G_CORE,0.001294261936049064,21516288.0,11.548010855130205,11.548010855130205,0.09900000000000375
2026-01-20T09:55:47,5G_CORE,0.000373588342440806,21520384.0,16.777552791437664,16.777552791437664,0.0
2026-01-20T09:56:03,5G_CORE,0.000373588342440806,21520384.0,0.0,0.0,0.0
2026-01-20T09:56:19,5G_CORE,8.577533577534192e-05,21524480.0,14.142891977566448,14.142891977566448,0.0
2026-01-20T09:57:08,5G_CORE,1.7192958421809795e-05,21528576.0,14.532771399505885,14.532771399505885,0.0
2026-01-20T09:57:24,5G_CORE,0.0,21528576.0,9.91707275369753,9.91707275369753,0.0
2026-01-20T09:57:40,5G_CORE,0.0024309000489947612,21532672.0,131.891433418151,131.891433418151,0.36599999999999966
2026-01-20T09:57:56,5G_CORE,0.0,21532672.0,241.0272471030379,241.0272471030379,0.44100000000000025
2026-01-20T09:58:13,5G_CORE,0.005920939514891015,21532672.0,459.16257430860685,459.16257430860685,2.13000000000000026
2026-01-20T09:58:29,5G_CORE,0.0024418925579103162,21532672.0,0.0,0.0,0.85199999999999996
2026-01-20T09:59:17,5G_CORE,0.008413189572409734,21544960.0,0.0,0.0,3.186
2026-01-20T09:59:49,5G_CORE,0.005430036188178518,21544960.0,445.9078080903104,445.9078080903104,5.07
2026-01-20T10:00:05,5G_CORE,0.007274965126774429,21549056.0,0.0,0.0,2.29200000000000016
2026-01-20T10:00:21,5G_CORE,0.005973044049967119,21553152.0,0.0,0.0,1.54500000000000017
2026-01-20T10:00:37,5G_CORE,0.0030875015828795904,21553152.0,492.271505376344,488.35125444802867,1.98899999999999972
```



System Architecture (The Custom Plugin)

- Architecture: Out-of-tree Kubernetes Scheduling Framework Plugin.
- Language: Go (Golang) compiled natively with CGO.
- Inference Engine: onnxruntime_go executing the ML model in milliseconds.
- Real-Time Integration: Local IPv4 port-forwarding to Prometheus for live cluster state.

Out-of-Tree Kubernetes Custom Scheduler



Dynamic Workload Awareness

- The Challenge: Energy scales dynamically with traffic, not just static YAML requests.
- The Solution: Live Cohort Averaging.
- Mechanism: The Go binary dynamically queries Prometheus (container_network_receive_bytes_total) to find the real-time average traffic of similar pods.
- Result: The ONNX model recalculates the predicted power cost based on live network congestion.

```
I0222 17:38:53.138794 122715 eventhandlers.go:128] "Add event for unscheduled pod" pod="default/test-green-pod"
I0222 17:38:53.144284 122715 schedule_one.go:84] "About to try and schedule pod" pod="default/test-green-pod"
I0222 17:38:53.144303 122715 schedule_one.go:97] "Attempting to schedule pod" pod="default/test-green-pod"
I0222 17:38:53.323933 122715 log.go:244] Pod: test-green-pod | Node: masterserver | NodePower: 0.00W | PodCost: 317.03 | Score: 68
I0222 17:38:53.353005 122715 log.go:244] Pod: test-green-pod | Node: worker2 | NodePower: 0.00W | PodCost: 317.03 | Score: 68
I0222 17:38:53.380054 122715 log.go:244] Pod: test-green-pod | Node: workerserver | NodePower: 0.00W | PodCost: 317.03 | Score: 68
I0222 17:38:53.399717 122715 trace.go:236] Trace[1734845985]: "Scheduling" namespace:default,name:test-green-pod (22-Feb-2026 17:38:53.144) (total time: 255ms):
ace[1734845985]: ---"Snapshotting scheduler cache and node infos done" 0ms (17:38:53.144)
ace[1734845985]: ---"Computing predicates done" 2ms (17:38:53.147)
Trace[1734845985]: ---"Prioritizing done" 252ms (17:38:53.399)
Trace[1734845985]: [255.360738ms] [255.360738ms] END
I0222 17:38:53.399883 122715 default_binder.go:53] "Attempting to bind pod to node" logger="Bind.DefaultBinder" pod="default/test-green-pod" node="worker2"
I0222 17:38:53.447772 122715 eventhandlers.go:175] "Delete event for unscheduled pod" pod="default/test-green-pod"
I0222 17:38:53.447977 122715 schedule_one.go:304] "Successfully bound pod to node" pod="default/test-green-pod" node="worker2" evaluatedNodes=3 feasibleNodes=3
I0222 17:38:53.448143 122715 eventhandlers.go:202] "Add event for scheduled pod" pod="default/test-green-pod"
```

Fig 1: Scheduler without Prometheus pipeline: scoring doesn't work

```
I0222 17:58:00.167973 128784 schedule_one.go:84] "About to try and schedule pod" pod="default/test-green-pod"
I0222 17:58:00.167980 128784 schedule_one.go:97] "Attempting to schedule pod" pod="default/test-green-pod"
I0222 17:58:00.288473 128784 log.go:244] Pod: test-green-pod | Node: workerserver | NodePower: 83.87W | PodCost: 317.03 | Score: 84
I0222 17:58:00.288594 128784 log.go:244] Pod: test-green-pod | Node: masterserver | NodePower: 117.52W | PodCost: 317.03 | Score: 59
I0222 17:58:00.288991 128784 log.go:244] Pod: test-green-pod | Node: worker2 | NodePower: 80.29W | PodCost: 317.03 | Score: 88
I0222 17:58:00.311384 128784 trace.go:236] Trace[1943436023]: "Scheduling" namespace:default,name:test-green-pod (22-Feb-2026 17:58:00.167) (total time: 143ms):
Trace[1943436023]: ---"Snapshotting scheduler cache and node infos done" 0ms (17:58:00.168)
Trace[1943436023]: ---"Computing predicates done" 0ms (17:58:00.168)
Trace[1943436023]: ---"Prioritizing done" 143ms (17:58:00.311)
Trace[1943436023]: [143.350129ms] [143.350129ms] END
I0222 17:58:00.313759 128784 default_binder.go:53] "Attempting to bind pod to node" logger="Bind.DefaultBinder" pod="default/test-green-pod" node="worker2"
I0222 17:58:00.339355 128784 schedule_one.go:304] "Successfully bound pod to node" pod="default/test-green-pod" node="worker2" evaluatedNodes=3 feasibleNodes=3
```

Fig 2: Scheduler with Prometheus pipeline: node energy, scoring

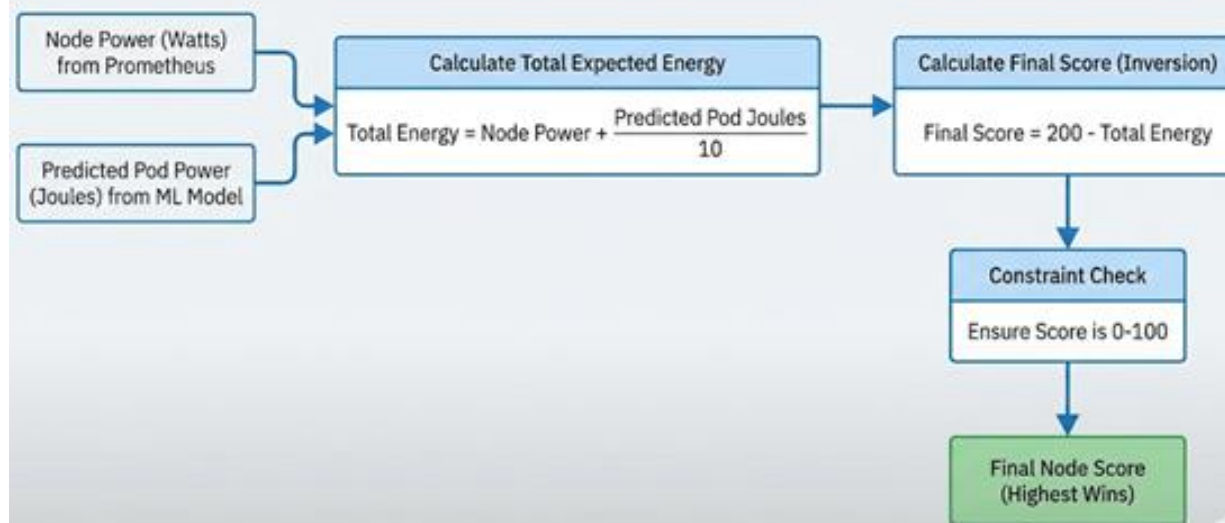
```
I0222 21:02:09.279278 170553 server.go:217] "Handlers synced"
I0222 21:02:09.279278 170553 eventhandlers.go:93] "Update event for node" node="masterserver"
I0222 21:02:09.279278 170553 eventhandlers.go:93] "Update event for node" node="workerserver"
I0222 21:02:09.279278 170553 eventhandlers.go:128] "Add event for unscheduled pod" pod="default/test-srsran-pod"
I0222 21:02:09.279278 170553 schedule_one.go:84] "About to try and schedule pod" pod="default/test-srsran-pod"
I0222 21:02:09.279278 170553 schedule_one.go:97] "Attempting to schedule pod" pod="default/test-srsran-pod"
I0222 21:02:09.279278 170553 log.go:244] Pod: test-srsran-pod | Node: worker2 | NodePower: 80.24W | NetRX: 2393808.00 | NetTX: 2393131.00 | PodCost: 312.29 | Score: 88
I0222 21:02:09.279278 170553 log.go:244] Pod: test-srsran-pod | Node: workerserver | NodePower: 153.29W | NetRX: 2393808.00 | NetTX: 2393131.00 | PodCost: 312.29 | Score: 15
I0222 21:02:09.332735 170553 log.go:244] Pod: test-srsran-pod | Node: masterserver | NodePower: 109.31W | NetRX: 2393808.00 | NetTX: 2393131.00 | PodCost: 312.29 | Score: 59
I0222 21:02:09.364595 170553 trace.go:236] Trace[382600078]: "Scheduling" namespace:default,name:test-srsran-pod (22-Feb-2026 21:02:08.881) (total time: 483ms):
Trace[382600078]: ---"Snapshotting scheduler cache and node infos done" 0ms (21:02:08.881)
Trace[382600078]: ---"Computing predicates done" 17ms (21:02:08.898)
Trace[382600078]: ---"Prioritizing done" 465ms (21:02:09.364)
Trace[382600078]: [483.200139ms] [483.200139ms] END
I0222 21:02:09.371301 170553 default_binder.go:53] "Attempting to bind pod to node" logger="Bind.DefaultBinder" pod="default/test-srsran-pod" node="worker2"
I0222 21:02:09.473326 170553 schedule_one.go:304] "Successfully bound pod to node" pod="default/test-srsran-pod" node="worker2" evaluatedNodes=3 feasibleNodes=3
I0222 21:02:09.475499 170553 eventhandlers.go:175] "Delete event for unscheduled pod" pod="default/test-srsran-pod"
I0222 21:02:09.475608 170553 eventhandlers.go:202] "Add event for scheduled pod" pod="default/test-srsran-pod"
I0222 21:02:09.635431 170553 eventhandlers.go:223] "Update event for scheduled pod" pod="default/test-srsran-pod"
I0222 21:02:10.278204 170553 eventhandlers.go:223] "Update event for scheduled pod" pod="default/test-srsran-pod"
```

Fig 3: node energy, network traffic, scoring and pod assignment

The Mathematics of Green Scoring

- The K8s Constraint: Score plugins must return an integer between 0 and 100 (Highest score wins).
- The Prediction: Total Energy = Node Power (W) + Predicted Pod Joules
- The Inversion: Final Score = 200 - Total Energy
- Fallback: Safety nets ensure anomalous readings default safely without crashing the scheduling cycle.

Mathematics of Green Scoring



Execution & Results

- The K8s Constraint: Score plugins must return an integer between 0 and 100 (Highest score wins).
- Kepler Querying: Real-time power is fetched using PromQL. query := `fmt.Sprintf(`sum by (exported_instance) (rate(kepler_node_platform_joules_total{exported_instance="%s"}[1m]))`, nodeName)`
- The Inversion: Final Score = 200 - Total Energy
- Fallback: Safety nets ensure anomalous readings default safely without crashing the scheduling cycle.

```
master@masterServer:~/ML_Integration_K8s_Scheduler/testPods$ cat test-green-pod.yaml
apiVersion: v1
kind: Pod
metadata:
  # name: test-green-pod
  name: test-srsran-pod
  namespace: default
spec:
  schedulerName: GreenScheduler # This routes the pod to custom binary
  containers:
  - name: test-container
    image: nginx
    resources:
      requests:
        cpu: "500m"
        memory: "512Mi"
```

```
I0222 21:00:24.989242 170553 server.go:217] "Handlers synced"
I0222 21:00:31.491107 170553 eventhandlers.go:93] "Update event for node" node="masterserver"
I0222 21:00:51.542523 170553 eventhandlers.go:93] "Update event for node" node="workerserver"
I0222 21:02:08.879141 170553 eventhandlers.go:128] "Add event for unscheduled pod" pod="default/test-srsran-pod"
I0222 21:02:08.881333 170553 schedule_one.go:84] "About to try and schedule pod" pod="default/test-srsran-pod"
I0222 21:02:08.881342 170553 schedule_one.go:97] "Attempting to schedule pod" pod="default/test-srsran-pod"
I0222 21:02:09.240043 170553 log.go:244] Pod: test-srsran-pod | Node: worker2 | NodePower: 80.24W | NetRX: 2393808.00 | NetTX: 2393131.00 | PodCost: 312.29 | Score: 88
I0222 21:02:09.279278 170553 log.go:244] Pod: test-srsran-pod | Node: workerserver | NodePower: 153.29W | NetRX: 2393808.00 | NetTX: 2393131.00 | PodCost: 312.29 | Score: 15
I0222 21:02:09.332735 170553 log.go:244] Pod: test-srsran-pod | Node: masterserver | NodePower: 109.31W | NetRX: 2393808.00 | NetTX: 2393131.00 | PodCost: 312.29 | Score: 59
I0222 21:02:09.364595 170553 trace.go:236] Trace[382600078]: "Scheduling" namespace:default,name:test-srsran-pod (22-Feb-2026 21:02:08.881) (total time: 48 3ms):
Trace[382600078]: ---"Snapshotting scheduler cache and node infos done" 0ms (21:02:08.881)
Trace[382600078]: ---"Computing predicates done" 17ms (21:02:08.898)
Trace[382600078]: ---"Prioritizing done" 465ms (21:02:09.364)
Trace[382600078]: [483.200139ms] [483.200139ms] END
I0222 21:02:09.371301 170553 default_binder.go:53] "Attempting to bind pod to node" logger="Bind.DefaultBinder" pod="default/test-srsran-pod" node="worker2"
I0222 21:02:09.473326 170553 schedule_one.go:304] "Successfully bound pod to node" pod="default/test-srsran-pod" node="worker2" evaluatedNodes=3 feasibleNodes=3
I0222 21:02:09.475499 170553 eventhandlers.go:175] "Delete event for unscheduled pod" pod="default/test-srsran-pod"
I0222 21:02:09.475608 170553 eventhandlers.go:202] "Add event for scheduled pod" pod="default/test-srsran-pod"
I0222 21:02:09.635431 170553 eventhandlers.go:223] "Update event for scheduled pod" pod="default/test-srsran-pod"
I0222 21:02:10.278204 170553 eventhandlers.go:223] "Update event for scheduled pod" pod="default/test-srsran-pod"
```

Deployment & Conclusion

- Production Readiness: Packaged via Dockerfile (Ubuntu base + compiled Go + ONNX .so libraries).
- Security: Deployed to kube-system with strict RBAC system:kube-scheduler bindings.
- Conclusion: Successfully validated ML energy models and engineered an active, self-healing orchestration engine that reduces carbon footprints without human intervention.

